

# ChessVerse: Comprehensive Code & Architecture Documentation

## Chapter 1 : Introduction

This document serves as a comprehensive technical guide to the ChessVerse application. It explains every major file in the project, what it was made for, and how the codebase works together to deliver a real-time chess tournament experience.

## Chapter 2 : Root Configurations & Files

- package.json / package-lock.json: These files define the project's dependencies (such as React, Firebase, framer-motion, lucide-react) and scripts. We use Vite (npm run dev) for ultra-fast module bundling and hot-reloading.
- index.html: The core HTML file. It mounts the `<div id='root'></div>` where the React application injects itself. It also links the favicon (chesslogo.jpeg).
- vite.config.js: Configuration file for the Vite bundler, utilizing the `@vitejs/plugin-react` to compile JSX and ES modules.
- vercel.json: Configuration for deploying the frontend to Vercel, ensuring client-side routing works by rewriting all traffic to index.html.

## Chapter 3 : Frontend Routing (App.jsx & main.jsx)

- src/main.jsx: The entry point of the React app. It imports the global CSS (index.css) and renders the `<App />` component into the root DOM node using React 19's `createRoot`.
- src/App.jsx: The routing hub. It utilizes `react-router-dom` to define paths like `'/'` (LandingPage), `'/chess-owner-entry'` (Host Dashboard Login), and `'/chess-viewer-room'` (Live Spectator). It also wraps everything in an `<ErrorBoundary>` component to catch rendering errors.

## Chapter 4 : Styling & Assets

- src/index.css & src/App.css: Contains all the global styling variables. We recently executed a massive UI renovation, removing AI-generated 'slop' and replacing it with a custom Premium Light Theme. It uses subtle glassmorphism (backdrop-filter), modern elevation shadows, and a radial background gradient.
- public/raj.jpg & logo.jpg: Static assets served at the root URL. Used in the About section and the Landing Page header.

## Chapter 5 : Core Pages: Landing & Entries

- src/pages/LandingPage.jsx: The face of ChessVerse. It contains the premium navigation bar with functional 'Tournaments', 'Rules', and 'About' modals. It utilizes Framer Motion to animate the FIDE Standard Rules modal and the Developer Profile Card (Rajtilak Chamlagain).
- src/pages/ChessOwnerEntry.jsx: The authentication gatekeeper. It ensures only authorized personnel (via Google Auth) can create or manage a tournament room. It generates the 6-digit Host Code.
- src/pages/ChessPlayerEntry.jsx: The player registration flow. Players input their name and a valid

# ChessVerse: Comprehensive Code & Architecture Documentation

tournament room code. The app validates the code in Firestore and injects their document into the room's 'players' collection.

- `src/pages/ChessViewerEntry.jsx`: The spectator gateway. It also fetches 'Recent Tournaments' from Firestore, allowing viewers to jump into ongoing matches.

## Chapter 6 : Core Pages: Real-time Dashboards

- `src/pages/ChessDashboard.jsx`: The massive Host Dashboard. This file connects directly to Firebase to manage the tournament state. When the host clicks 'Generate Next Round', it runs the pairing algorithms and pushes the new matchups to Firestore. The host can input match scores (Win/Draw/Loss), and the dashboard updates the players' overall points and tie-breakers immediately.

- `src/pages/ChessViewerRoom.jsx`: The Live Spectator Hub. This component sets up a Firebase 'onSnapshot' listener. This means it establishes a continuous WebSocket connection to the database. Whenever the Host updates a score in `ChessDashboard.jsx`, this component receives the new data in milliseconds and instantly re-renders the live leaderboard and match results without the user ever refreshing the page.

## Chapter 7 : Backend Integration (Firebase)

- `src/firebase.js`: Initializes the Firebase app with the required API keys. It exports the 'auth' object (for Google Sign-in) and the 'db' object (Firestore). Firestore acts as our NoSQL database, storing tournaments as documents, with 'players' and 'rounds' structured as sub-collections.

## Chapter 8 : Development Scripts (Python & JS)

Throughout the development process, multiple Python and Node scripts (e.g., `update_jsx.py`, `fix_codes.py`, `full_renovation.py`) were created to automate massive codebase refactors. Instead of manually editing 5,000 lines of code to switch themes or fix missing imports, these scripts utilized regex and AST manipulation to automatically patch files, saving hours of manual labor.